CASE STUDY 3: STUDENT PERFORMANCE TREND ANALYSIS

1. Problem Statement

Student Performance Trend Analysis is used to analyze a student's marks in different examinations and identify whether the student's performance is improving, declining, or remaining constant. The program accepts marks from the user, calculates the average, and compares consecutive examination marks to identify the performance trend.

2. Objectives

- To develop a Python program to analyze student performance.
- To calculate the average marks of a student.
- To identify improving, declining, or constant performance.
- To use lists, loops, and conditional statements in Python.
- To understand how performance trends can be analyzed using programming.

3. Assumptions

1. The student provides marks for three examinations.
2. Each examination mark must be between 0 and 100.
3. Marks are entered in chronological order.
4. The average is calculated using all three examination marks.
5. The trend is determined by comparing consecutive examination marks.

4. Algorithm

1. Start the program.
2. Accept marks for three examinations from the user.
3. Check whether all marks are between 0 and 100.
4. If any mark is invalid, display an error message.
5. Otherwise, calculate the average of the three marks.
6. Compare the second examination mark with the first examination mark.
7. Compare the third examination mark with the second examination mark.
8. If both comparisons show an increase, display Improving Performance.
9. If both comparisons show a decrease, display Declining Performance.
10. If all three marks are equal, display Constant Performance.
11. Otherwise, display Fluctuating Performance.
12. Display the average marks.
13. Stop the program.

5. Flowchart

Start

↓

Enter marks for three examinations

↓

Check whether all marks are between 0 and 100

↓

Are the marks valid?

- No → Display Invalid Marks → Stop
- Yes → Calculate Average

↓

Compare consecutive examination marks

↓

- Both comparisons increase → Improving Performance
- Both comparisons decrease → Declining Performance
- All marks are equal → Constant Performance
- Otherwise → Fluctuating Performance

↓

Display average and performance trend

↓

Stop

6. Pseudocode

```
START

INPUT mark1, mark2, mark3

IF any mark < 0 OR any mark > 100
   DISPLAY "Invalid Marks"

ELSE
   average = (mark1 + mark2 + mark3) / 3
```

```
    IF mark1 < mark2 AND mark2 < mark3
        DISPLAY "Improving Performance"

    ELSE IF mark1 > mark2 AND mark2 > mark3
        DISPLAY "Declining Performance"

    ELSE IF mark1 = mark2 AND mark2 = mark3
        DISPLAY "Constant Performance"

    ELSE
        DISPLAY "Fluctuating Performance"
    END IF

    DISPLAY average
END IF

STOP
```

7. Python Source Code

```
marks = []

for i in range(1, 4):
    mark = int(input("Enter marks for exam " + str(i) + ": "))
    marks.append(mark)

if any(mark < 0 or mark > 100 for mark in marks):
    print("Invalid Marks")

else:
    average = sum(marks) / 3

    print("Average Marks:", average)

    if marks[0] < marks[1] < marks[2]:
        print("Performance Trend: Improving")

    elif marks[0] > marks[1] > marks[2]:
        print("Performance Trend: Declining")

    elif marks[0] == marks[1] == marks[2]:
        print("Performance Trend: Constant")

    else:
        print("Performance Trend: Fluctuating")
```

8. Test Cases and Outputs

Test Case 1: Improving Performance

Input:

Enter marks for exam 1: 60
Enter marks for exam 2: 70
Enter marks for exam 3: 85

Expected Output:

Average Marks: 71.66666666666667
Performance Trend: Improving

Test Case 2: Declining Performance

Input:

Enter marks for exam 1: 90
Enter marks for exam 2: 75
Enter marks for exam 3: 60

Expected Output:

Average Marks: 75.0
Performance Trend: Declining

Test Case 3: Invalid Marks

Input:

Enter marks for exam 1: 80
Enter marks for exam 2: 110
Enter marks for exam 3: 70

Expected Output:

Invalid Marks

Note: Run each test case separately. The program accepts three examination marks for every execution.

9. Output Screenshots

```python
marks = []

for i in range(1, 4):
    mark = int(input("Enter marks for exam " + str(i) + ": "))
    marks.append(mark)

if any(mark < 0 or mark > 100 for mark in marks):
    print("Invalid Marks")

else:
    average = sum(marks) / 3

    print("Average Marks:", average)

    if marks[0] < marks[1] < marks[2]:
        print("Performance Trend: Improving")

    elif marks[0] > marks[1] > marks[2]:
        print("Performance Trend: Declining")

    elif marks[0] == marks[1] == marks[2]:
        print("Performance Trend: Constant")

    else:
        print("Performance Trend: Fluctuating")
```

```
Python 3.15.0 (tags/v3.15.0:d220409, Oct  9 2026, 11:55:56) [MSC v.1951 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>
= RESTART: C:/Users/sirit/OneDrive/Desktop/Python-Activity-Based-Learning/case3.
py
Enter marks for exam 1: 60
Enter marks for exam 2: 70
Enter marks for exam 3: 85
Average Marks: 71.66666666666667
Performance Trend: Improving
>>
= RESTART: C:/Users/sirit/OneDrive/Desktop/Python-Activity-Based-Learning/case3.
py
Enter marks for exam 1: 90
Enter marks for exam 2: 75
Enter marks for exam 3: 60
Average Marks: 75.0
Performance Trend: Declining
>>
= RESTART: C:/Users/sirit/OneDrive/Desktop/Python-Activity-Based-Learning/case3.
py
Enter marks for exam 1: 80
Enter marks for exam 2: 110
Enter marks for exam 3: 70
Invalid Marks
>>
```

10. Conclusion

Thus, the Student Performance Trend Analysis program was successfully implemented using Python. The program calculates average marks and identifies improving, declining, constant, or fluctuating performance. It also validates the marks entered by the user. This case study demonstrates the use of lists, loops, conditional statements, and basic arithmetic operations in Python.